

PYTHON FUNDAMENTALS

Variables & Core Data Types

An executive-grade training handbook

Variables · Object Model · Memory · Numbers · Strings · Booleans · Conversion · Hashability

FORMAT

Self-paced / Instructor-led

LEVEL

Foundational

SERIES

Session 02

Executive Summary

This handbook explains how Python stores and labels data. Mastering it removes the most common source of beginner bugs: confusing a *name* with the *object* it points to. Every later topic — collections, functions, classes — rests on these foundations.

The material is organised into nine sections that move from how names bind to objects, through the object and memory model, into the core data types (numbers, strings, booleans), and finally to type conversion, identification, and hashability.

Executive Insight

Treat variables as labels on objects, never as containers of values. This single mental model explains assignment, copying, mutation, identity, and most “surprising” Python behaviour.

Each concept follows a consistent rhythm — What it is, Why it matters, a runnable Example with expected Output, and a Key Takeaway — so the document is equally suited to first-time learning and later review.

Table of Contents

Executive Summary.....	2
I. Variable Fundamentals	4
A. Understanding Variables.....	4
B. Assignment Techniques.....	5
C. Identifiers & Keywords	6
II. The Python Object Model.....	8
A. Objects & References	8
B. Introspection Tools.....	9
C. Object Representation	9
III. Memory Management	10
A. Storage Model.....	10
B. Object Lifecycle.....	10
IV. Numeric Types.....	12
A. Integers (int).....	12
B. Floating-Point (float).....	12
V. Strings	14
A. String Literals	14
B. Escape Sequences	14
C. String Behaviour	14
VI. Booleans & None.....	16
A. Boolean Logic	16
B. None vs False vs 0.....	16
VII. Type Conversion & Identification	17
A. Type Conversion.....	17
B. Type Identification	17
C. Safe Input Handling	18
VIII. Hashability	19
A. Hashing	19
IX. Documentation	20
A. Docstrings	20
Best Practices Checklist	21
Common Mistakes	22
Key Takeaways at a Glance	23
Summary & Next Steps	24

I. Variable Fundamentals

How names attach to objects, and every practical way to assign, rebind, and remove them.

A. Understanding Variables

1. What is a Variable?

What it is — A name that refers to an object stored in memory. Python names hold references, not the values themselves.

Why it matters — It explains all later behaviour around copying, mutation, and identity.

Important Note

No declaration is required: a name is created the instant you assign to it.

Python Example	Output
<pre>name = "Arun" age = 25 print(name) print(age)</pre>	<pre>Arun 25</pre>

Key Takeaway A variable is a label on an object — not a box that contains it.

2. Creating Variables & Assignment (=)

What it is — The = operator binds a name to an object; the variable is created on first assignment.

Why it matters — It is the single most common operation you will write.

Python Example	Output
<pre>city = "Bengaluru" x = 100 print(city) print(x)</pre>	<pre>Bengaluru 100</pre>

Key Takeaway Assignment is name → object binding, not value storage.

3. Dynamic Typing

What it is — A name may reference different types at different times; the type lives on the object, not the name.

Why it matters — Enables flexible code, but means types can change silently.

Python Example	Output
<pre>data = 10 print(type(data)) data = "Python" print(type(data))</pre>	<pre><class 'int'> <class 'str'></pre>

Key Takeaway Variables are untyped; objects are typed.

B. Assignment Techniques

1. Reassignment

What it is — Pointing an existing name at a new object.

Why it matters — Program state changes are just reassignments.

Python Example	Output
<pre>score = 50 score = 75 print(score)</pre>	75

Key Takeaway Reassigning rebinds the name; the old object is left for cleanup.

2. Multiple & Chain Assignment

What it is — Bind several names at once ($x, y = 1, 2$) or bind many names to one object ($a = b = c = 5$).

Why it matters — Cleaner initialisation and setup code.

Python Example	Output
<pre>x, y, z = 10, 20, 30 print(x, y, z) a = b = c = 5 print(a, b, c)</pre>	<pre>10 20 30 5 5 5</pre>

Key Takeaway Multiple assignment unpacks values; chain assignment shares one object.

3. Variable Swapping

What it is — Exchange two values in one line with no temporary variable.

Why it matters — A clean, Pythonic idiom used constantly.

Python Example	Output
<pre>x = 10 y = 20 x, y = y, x print(x, y)</pre>	20 10

Key Takeaway $x, y = y, x$ swaps in place — no temp variable needed.

4. Augmented Assignment ($+=$)

What it is — $x += 1$ is shorthand, but for mutable objects it modifies in place while $x = x + 1$ always rebinds.

Why it matters — The difference quietly causes shared-list bugs.

Executive Insight

Watch the identity check: the list keeps its id after $+=$, but the integer gets a brand-new one.

Python Example	Output
<pre>lst = [1, 2] before = id(lst) lst += [3] # mutates in place</pre>	<pre>[1, 2, 3] True 6 False</pre>

```
print(lst, id(lst) == before)

n = 5
before = id(n)
n += 1      # rebinds to new int
print(n, id(n) == before)
```

Key Takeaway For immutables the two match; for lists, += mutates the same object.

5. Deleting Variables (del)

What it is — del removes a name from the namespace; it does not directly destroy the object.

Why it matters — Frees names and lets unreferenced objects be reclaimed.

Python Example	Output
<pre>name = "Python" del name try: print(name) except NameError as e: print("NameError:", e)</pre>	<pre>NameError: name 'name' is not defined</pre>

Key Takeaway del unbinds a name; using it afterwards raises NameError.

C. Identifiers & Keywords

1. Naming Rules & Best Practices

What it is — Identifiers use letters, digits, and underscores, cannot start with a digit, and are case-sensitive.

Why it matters — Good names are the cheapest documentation you can write.

Python Example	Output
<pre>total_revenue = 1500 customer_count = 42 print(total_revenue, customer_count) print("Case-sensitive:", "Total" != "total")</pre>	<pre>1500 42 Case-sensitive: True</pre>

Best Practice

total_revenue beats tr; customer_count beats c. Names should reveal intent without a comment.

Key Takeaway Use descriptive snake_case; reserve single letters for short-lived counters.

2. Reserved Keywords

What it is — Words reserved by the language (for, class, None...) that cannot be used as names.

Why it matters — Using one as a variable is an instant SyntaxError.

Python Example	Output
<pre>import keyword print("Total keywords:", len(keyword.kwlist)) print(keyword.iskeyword("class")) print(keyword.iskeyword("data"))</pre>	<pre>Total keywords: 35 True False</pre>

Key Takeaway Check membership with the keyword module instead of memorising the list.

II. The Python Object Model

In Python everything is an object, and names are simply references to those objects.

A. Objects & References

1. Everything is an Object

What it is — Integers, strings, functions, and classes are all objects, each with a type.

Why it matters — One consistent model explains the entire language.

Python Example	Output
<pre>x = 10 name = "Python" print(type(x)) print(type(name)) print(type(print))</pre>	<pre><class 'int'> <class 'str'> <class 'builtin_function_or_method'></pre>

Key Takeaway If it exists in Python, it is an object.

2. Identity, Type & Value

What it is — Every object has an identity (id), a type (type), and a value.

Why it matters — These three properties define what an object is.

Python Example	Output
<pre>x = 100 print("type:", type(x)) print("value:", x) print("id:", id(x)) # varies per run</pre>	<pre>type: <class 'int'> value: 100 id: 11758856</pre>

Important Note

id() values vary by machine and run — only equality between two ids is meaningful, never the number itself.

Key Takeaway Identity = address, Type = kind, Value = content.

3. Variables as References

What it is — Two names assigned to the same object share one identity until one is rebound.

Why it matters — This is why mutating through one name is visible through the other.

Python Example	Output
<pre>x = [1, 2, 3] y = x print(x is y) y.append(4) print("x sees it:", x)</pre>	<pre>True x sees it: [1, 2, 3, 4]</pre>

Key Takeaway y = x copies the reference, not the object. Use “is” for identity, “==” for value.

B. Introspection Tools

1. `type()`, `id()`, `dir()`, `help()`

What it is — The core inspection toolkit: kind, identity, available members, and live documentation.

Why it matters — They let you investigate any object without leaving the editor.

Python Example	Output
<pre>s = "Python" print(type(s)) print([m for m in dir(s) if not m.startswith("_")][:4]) print(len.__doc__)</pre>	<pre><class 'str'> ['capitalize', 'casefold', 'center', 'count'] Return the number of items in a container.</pre>

Key Takeaway type = what, id = where, dir = what-it-can-do, help = how.

C. Object Representation

1. `str()` vs `repr()`

What it is — `str()` gives a readable form for users; `repr()` gives an unambiguous form for developers.

Why it matters — Logging and debugging depend on `repr()` showing the real structure.

Python Example	Output
<pre>s = "line1\nline2" print("str :", str(s)) print("repr:", repr(s))</pre>	<pre>str : line1 line2 repr: 'line1\nline2'</pre>

Key Takeaway str = friendly; repr = precise (note the quotes and visible escapes).

III. Memory Management

Where objects live, how Python tracks them, and when it reclaims them.

A. Storage Model

1. References & Object Size

What it is — A name holds a reference; `sys.getsizeof()` reports an object's footprint in bytes.

Why it matters — Useful when profiling memory-heavy data.

Python Example	Output
<pre>import sys print("int :", sys.getsizeof(100)) print("str :", sys.getsizeof("Python")) print("list:", sys.getsizeof([1, 2, 3]))</pre>	<pre>int : 28 str : 47 list: 88</pre>

Important Note

Exact byte counts depend on the Python build and platform; compare relative sizes, not absolutes.

Key Takeaway Richer objects cost more bytes — measure, don't guess.

2. Stack vs Heap (Conceptual)

What it is — References conceptually live on the stack; the actual objects live on the heap.

Why it matters — Explains why assigning a name is cheap even for huge objects.

Python Example	Output
<pre>x = 10 # name on stack -> int object on heap y = x # second pointer to SAME object print(id(x) == id(y))</pre>	<pre>True</pre>

Key Takeaway Names are lightweight pointers; the data sits on the heap.

B. Object Lifecycle

1. Reference Counting

What it is — CPython keeps a count of references to each object (`sys.getrefcount()`).

Why it matters — When the count reaches zero, the object can be freed.

Python Example	Output
<pre>import sys obj = [] print("refs:", sys.getrefcount(obj)) alias = obj print("after alias:", sys.getrefcount(obj))</pre>	<pre>refs: 2 after alias: 3</pre>

Important Note

Python Example	Output
The count reads one higher than expected because the argument passed to <code>getrefcount()</code> is itself a temporary reference.	

Key Takeaway More references keep an object alive longer.

2. Garbage Collection

What it is — Python automatically reclaims objects with no remaining references.

Why it matters — You manage logic; the runtime manages memory.

Python Example	Output
<pre>import gc x = [1, 2, 3] del x print("collected:", gc.collect())</pre>	collected: 6

Key Takeaway Drop the last reference and cleanup happens automatically.

IV. Numeric Types

Integers and floats — literals, digit separators, and the special values to watch for.

A. Integers (int)

1. Numeric Literals: Decimal, Binary, Octal, Hex

What it is — The same integer can be written in base 10, base 2 (0b), base 8 (0o), or base 16 (0x).

Why it matters — Bit masks, permissions, and colours are far clearer in their natural base.

Python Example	Output
<code>print(255)</code>	255
<code>print(0b11111111)</code>	255
<code>print(0o377)</code>	255
<code>print(0xFF)</code>	255
<code>print(255 == 0xFF == 0b11111111)</code>	True

Key Takeaway Different notations, one identical integer value.

2. Underscore Separators

What it is — Underscores group digits in large numeric literals and are ignored by Python.

Why it matters — Readability for big numbers with zero runtime cost.

Python Example	Output
<code>population = 1_500_000</code>	1500000
<code>print(population)</code>	True
<code>print(population == 1500000)</code>	

Key Takeaway 1_500_000 reads instantly; the stored value is plain 1500000.

B. Floating-Point (float)

1. Float Basics

What it is — Numbers with a decimal point or exponent; division always yields a float.

Why it matters — Most measured, scientific, and financial values are floats.

Python Example	Output
<code>print(type(3.14))</code>	<class 'float'>
<code>print(10 / 3)</code>	3.3333333333333335
<code>print(0.1 + 0.2)</code>	0.30000000000000004

Important Note

0.1 + 0.2 is famously not exactly 0.3. For money, use the decimal module instead of raw floats.

Key Takeaway Use floats for fractions — but expect tiny binary rounding error.

2. Special Float Values: inf & nan

What it is — Floats include positive/negative infinity and NaN (Not a Number).

Why it matters — They surface in overflow, undefined math, and missing data.

Python Example	Output
<pre>pos = float('inf') nan = float('nan') print(pos, float('-inf'), nan) print('inf > huge:', pos > 1e308) print('nan == nan:', nan == nan)</pre>	<pre>inf -inf nan inf > huge: True nan == nan: False</pre>

Key Takeaway nan is never equal to anything — even itself.

V. Strings

Text literals, escape sequences, comparison, encoding, and how Python reuses string objects.

A. String Literals

1. Quote Variants & f-strings

What it is — Single, double, or triple quotes; prefix `r` for raw (no escapes) or `f` for formatted strings.

Why it matters — The right quote style avoids escaping headaches and powers clean formatting.

Python Example	Output
<pre>triple = """line A line B""" raw = r"C:\new\path" name = "Arun" print(triple) print(raw) print(f"Hello {name}, score={40+2}")</pre>	<pre>line A line B C:\new\path Hello Arun, score=42</pre>

Key Takeaway Triple quotes span lines; `r"..."` keeps backslashes literal; `f"..."` embeds expressions.

B. Escape Sequences

1. Common Escape Sequences

What it is — Backslash codes insert special characters: `\n` `\t` `\r` `\b` `\\` `\'` `\"`.

Why it matters — They control layout and let you embed quotes safely.

Python Example	Output
<pre>print("a\tb") print("q: \"hi\"") print("back\\slash")</pre>	<pre>a b q: "hi" back\slash</pre>

Key Takeaway Most courses teach only `\n` — know the full set.

C. String Behaviour

1. String Comparison

What it is — Strings compare lexicographically by Unicode code point, and comparison is case-sensitive.

Why it matters — Sorting and searching depend on these exact rules.

Python Example	Output
<pre>print("apple" < "banana") print("Apple" < "apple") # A(65) < a(97) print("apple" == "Apple")</pre>	<pre>True True False</pre>

Key Takeaway Uppercase letters sort before lowercase because their code points are lower.

2. Character Encoding: `ord()` & `chr()`

What it is — `ord()` maps a character to its Unicode code point; `chr()` maps a code point back.

Why it matters — Foundational for encryption, hashing, and text processing.

Python Example	Output
<pre>print(ord('A')) print(chr(65)) print(chr(ord('A') + 1))</pre>	<pre>65 A B</pre>

Key Takeaway They are exact inverses: `chr(ord(c)) == c`.

3. String Interning (Pooling)

What it is — Python reuses one object for identical short string literals, so they share identity.

Why it matters — Saves memory and speeds some equality checks.

Python Example	Output
<pre>a = "python" b = "python" print("same object:", a is b) print("equal value :", a == b)</pre>	<pre>same object: True equal value : True</pre>

Important Note

Interning is an optimisation, not a guarantee. Never use `is` to compare string values.

Key Takeaway Equal literals may be the same object — but compare values with `==`, never with `is`.

VI. Booleans & None

Truthiness, boolean conversion, and the classic None / False / 0 confusion.

A. Boolean Logic

1. Truth Value Testing & bool()

What it is — Every object is truthy or falsy. Falsy values include 0, "" (empty string), empty containers, and None.

Why it matters — if statements test truthiness, not just True / False.

Python Example	Output
<pre>print(bool("")) print(bool("Python")) print(bool(0)) print(bool(-1)) print(bool([]))</pre>	<pre>False True False True False</pre>

Key Takeaway Empty / zero / None → falsy; almost everything else → truthy.

B. None vs False vs 0

1. Telling Them Apart

What it is — None means “no value”; False is a boolean; 0 is a number. False equals 0 by value, but None equals neither.

Why it matters — Confusing them produces subtle logic bugs.

Executive Insight

False == 0 is True because bool is a subclass of int. None is a distinct singleton and equals nothing else.

Python Example	Output
<pre>print("None == False:", None == False) print("None == 0 :", None == 0) print("False == 0 :", False == 0) print("None is None :", None is None)</pre>	<pre>None == False: False None == 0 : False False == 0 : True None is None : True</pre>

Key Takeaway Test for absence with “is None”, never with “== False”.

VII. Type Conversion & Identification

Convert safely between types, check types correctly, and validate user input.

A. Type Conversion

1. Core Conversions

What it is — `int()`, `float()`, and `str()` convert between types; `int()` truncates toward zero.

Why it matters — User input and file data arrive as strings and must be converted.

Python Example	Output
<pre>print(int("10") + 5) print(float("3.14")) print(int(3.9)) # truncates print(str(100) + "%")</pre>	<pre>15 3.14 3 100%</pre>

Key Takeaway Convert explicitly — Python will not mix strings and numbers for you.

2. Conversion Edge Cases

What it is — Booleans are integers under the hood, so `int(True)` is 1 and `bool(0)` is `False`.

Why it matters — These are classic interview questions and real bug sources.

Python Example	Output
<pre>print(int(True)) print(float(False)) print(bool(0)) print(bool(-1)) print(True + True)</pre>	<pre>1 0.0 False True 2</pre>

Key Takeaway `True` behaves as 1 and `False` as 0 in arithmetic.

B. Type Identification

1. `type()` vs `isinstance()`

What it is — `type()` checks for an exact type; `isinstance()` also accepts subclasses.

Why it matters — Choosing wrong breaks code that relies on inheritance.

Python Example	Output
<pre>print(isinstance(10, int)) print(isinstance(True, int)) # True! print(type(True) is int) # False print(isinstance("hi", (int, str)))</pre>	<pre>True True False True</pre>

Best Practice

Use `isinstance(x, int)` in production code; reserve `type(x) is int` for strict, exact-type checks.

Key Takeaway Prefer `isinstance()` for checks — note that `True` is also an `int`.

C. Safe Input Handling

1. Validate Before Converting

What it is — Check that text is numeric (e.g. `str.isdigit()`) before calling `int()` to avoid crashes.

Why it matters — Unvalidated `int(input())` raises `ValueError` on bad input.

Python Example	Output
<pre>def to_age(raw): if raw.isdigit(): return int(raw) return f"invalid: {raw!r}" print(to_age("25")) print(to_age("2a"))</pre>	<pre>25 invalid: '2a'</pre>

Key Takeaway Validate first, convert second — never trust raw input.

VIII. Hashability

The most-skipped data-type concept — and the key to sets and dictionary keys.

A. Hashing

1. hash(): Hashable vs Unhashable

What it is — Hashing maps an object to an integer. Immutable objects (numbers, strings, tuples) are hashable; mutable ones (lists, dicts) are not.

Why it matters — Only hashable objects can be dictionary keys or set members.

Executive Insight

Hashability tracks immutability: a mutable object cannot be hashable, because its hash would change underneath the collection.

Python Example	Output
<pre>print("hash tuple:", hash((1, 2, 3))) try: hash([1, 2, 3]) except TypeError as e: print("hash list :", e)</pre>	<pre>hash tuple: 529344067295497451 hash list : unhashable type: 'list'</pre>

Key Takeaway If you cannot hash() it, you cannot use it as a dictionary key.

IX. Documentation

Make objects self-describing with docstrings.

A. Docstrings

1. Docstrings & `__doc__`

What it is — A triple-quoted string as the first statement of a function, class, or module becomes its documentation.

Why it matters — It powers `help()`, editor tooltips, and auto-generated docs.

Python Example	Output
<pre>def to_celsius(f): """Convert Fahrenheit to Celsius.""" return (f - 32) * 5 / 9 print(to_celsius(98.6)) print(to_celsius.__doc__)</pre>	<pre>37.0 Convert Fahrenheit to Celsius.</pre>

Key Takeaway Document intent at the source; read it back through `__doc__`.

Best Practices Checklist

Carry these habits into every Python file.

- Name for intent — descriptive snake_case over abbreviations.
- Compare values with ==, identity with is; never use is on numbers or strings to compare value.
- Use isinstance() for type checks so subclasses (like bool) are handled correctly.
- Validate input before converting it; wrap risky conversions in try/except.
- Prefer the decimal module over float for money and exact arithmetic.
- Remember += mutates mutable objects in place — copy first if other names share the object.
- Document every function with a one-line docstring.

Common Mistakes

The errors that trip up almost every beginner — and the fix for each.

Common Mistake	Why It Goes Wrong	Correct Approach
Thinking a variable “contains” a value	Two names can point to one object, so changes appear in both	Treat names as labels; copy the object when you need independence
Using <code>is</code> to compare strings or numbers	Identity is not value; results depend on interning	Use <code>==</code> for value comparison
<code>int(input())</code> without validation	Crashes with <code>ValueError</code> on non-numeric input	Check <code>.isdigit()</code> or use <code>try/except</code> first
Assuming <code>+=</code> copies a list	<code>+=</code> mutates the shared list in place	Use <code>new_list = old + [item]</code> to keep them separate
Comparing to <code>False</code> or <code>0</code> for “no value”	<code>None</code> is not equal to <code>False</code> or <code>0</code>	Use <code>is None</code>
Trusting <code>0.1 + 0.2 == 0.3</code>	Binary floats cannot store every decimal exactly	Use <code>math.isclose()</code> or the <code>decimal</code> module
Using a list as a dictionary key	Lists are unhashable and raise <code>TypeError</code>	Use a tuple (immutable) as the key

Key Takeaways at a Glance

One line to remember from each section.

Section	Essence
I. Variable Fundamentals	Names are labels bound to objects, not containers of values.
II. Object Model	Everything is an object with an identity, type, and value.
III. Memory	Python tracks references and reclaims unreferenced objects automatically.
IV. Numbers	One integer value, many literal bases; floats carry tiny rounding error.
V. Strings	Choose the right quote style; compare by Unicode; use == not is.
VI. Booleans & None	Truthiness drives if; None is distinct from False and 0.
VII. Conversion	Convert explicitly and validate first; bool is a subclass of int.
VIII. Hashability	Only immutable objects can be dict keys or set members.
IX. Docstrings	Document intent at the source for living documentation.

Summary & Next Steps

You now command the foundations that every other Python topic builds on: how names bind to objects, how the object and memory model behave, and how the core data types — numbers, strings, booleans — convert, compare, and hash.

The single idea to keep is that **a variable is a reference, not a container**. Hold that, and assignment, mutation, identity, and copying all stop being surprising.

Best Practice

Next session — Strings & Lists: indexing, slicing, and the everyday methods that turn these foundations into working programs.